

3D Reconstruction and Robotic Manipulation

7CCEMSAP Coursework 1

Yufeng

1 Introduction

This project carries out the reconstruction of a 3D model of a soup can and uses it in a UR5e MoveIt simulation. The project timeline is listed as:

1. Video capture and frame extraction
2. Camera calibration
3. SIFT matching
4. Homography checks
5. SfM in ROS2
6. Dense reconstruction via VisualSFM
7. Mesh cleaning
8. MoveIt integration

There were several challenges that changed the development of the project. The use of 60fps video resulted in very small motion between consecutive frames, which resulted in a reduced geometric parallax. The ROS2 SfM implementation was sensitive to this and created unstable reconstructions, such as explosions and strange lines. Due to the lack of success and errors with ROS2 SfM, we resorted to using VisualSFM to produce a usable model. The mesh produced by VisualSFM had a blurred texture and lacked a bottom surface due to the floor, but the point cloud had a much better shape.

2 Image Capture and Calibration

A 1080x1920 60 fps video was recorded on an iPhone 17 Pro Max while moving around the object. `ffmpeg` was used to extract frames from the video by splicing the video equally to produce images for SfM. Because 60fps produces many nearly identical images, every- N th frame was extracted to produce the correct dataset for SfM.

For calibration, a printed 11x8 checkerboard was used with the ROS2 `cameracalibrator`, generating a YAML file containing intrinsic matrix K and distortion coefficients. The projection model:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} X/Z \\ Y/Z \\ 1 \end{bmatrix}$$

ensured the calibration was consistent throughout the project.

3 Feature Detection and Matching

SIFT features were extracted from all images and matched with Lowe’s ratio filtering. The feature matching performance was affected by the can’s repetitive vertical label patterns, cylindrical symmetry, and the inability to see under the can due to the floor. Homographies estimated via RANSAC,

$$x_2 \sim Hx_1,$$

showed the overlap between frames but was very inconsistent when the camera motion was minimal.

4 Structure from Motion in ROS2

The ROS2 SfM node estimated the essential matrix,

$$E = [t]_{\times} R,$$

followed by pose recovery (R, t) and triangulation using

$$P_1 = K[I|0], \quad P_2 = K[R|t].$$

Across multiple iterations, the reconstructed point clouds showed behaviour that matched some characteristics of the images in the dataset. The earlier attempts were overly dense and unstable, producing images such as scattered points and irregular structures due to weak essential matrix estimation and a lack of variation between frames (Figure 1). After reducing the number of images, the reconstruction became more numerically stable and produced cleaner curved surface strips that resembled parts of the can, although still incorrectly positioned (Figure 2). This behaviour occurs when the motion between frames is predominantly rotational and the object has strong cylindrical symmetry, leaving too little baseline for reliable depth estimation.

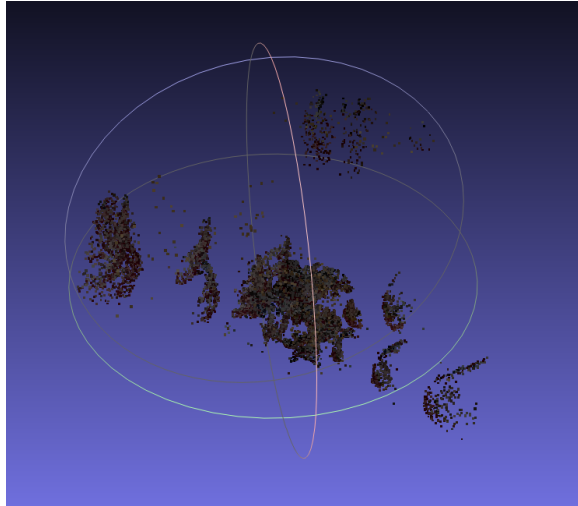


Figure 1: Early ROS2 SfM attempt: Very dense and reconstruction is unstable.

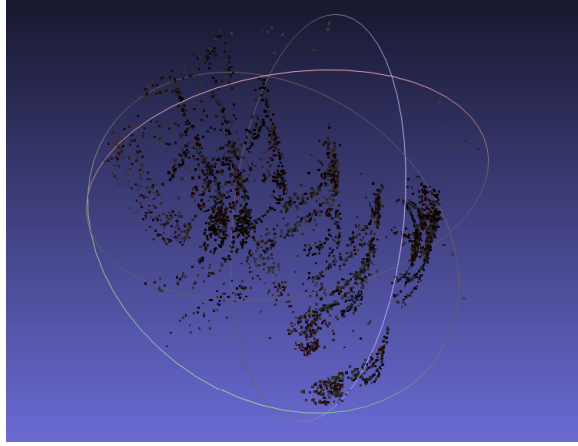


Figure 2: Later ROS2 SfM attempt: the number of images in the dataset is reduced, which produces cleaner curved surface fragments.

Due to the combined effects of low parallax, high frame rate, and cylindrical symmetry, the ROS2 SfM implementation was unable to reconstruct a complete and stable 3D model of the soup can.

5 Dense Reconstruction Using VisualSFM

To obtain a usable model for manipulation, the same dataset was processed using VisualSFM, which produced a denser and more coherent point cloud, due to its stronger feature matching and multi-view optimisation.

The point cloud was manually cleaned by removing the background, cropping around the object, running surface reconstruction, smoothing, filling holes, and exporting the final model as STL. The resulting mesh (Figure 4) captured the correct cylindrical geometry but contained blurred texture and lacked a bottom surface. These limitations reflect the original dataset: the bottom of the can was never visible, and the video contained some motion blur.

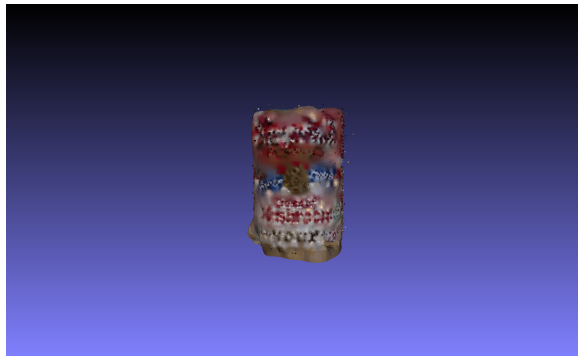


Figure 3: Cleaned VisualSFM STL mesh of the soup can used for MoveIt.

6 MoveIt Integration

The final cleaned soup can mesh was exported as an STL and added directly into Moveit. Once the UR5e configuration was launched, the STL was inserted at a fixed position, which allowed the robot to recognise it as a part of the environment.

With the mesh added, I tested both joint-space and Cartesian planning. Joint-space planning validated that the robot could move between predefined positions without violating joint limits

or causing self-collision. Cartesian planning was used to test the viability of moving the end-effector along a path around the object.

The OMPL RRTConnect planner produced valid collision-free trajectories in both modes, which shows that the MoveIt correctly recognised the mesh as a solid object and accounted for it during collision checking. This confirms that the planning scene was properly configured and path generation respected both kinematic constraints and the object’s geometry.

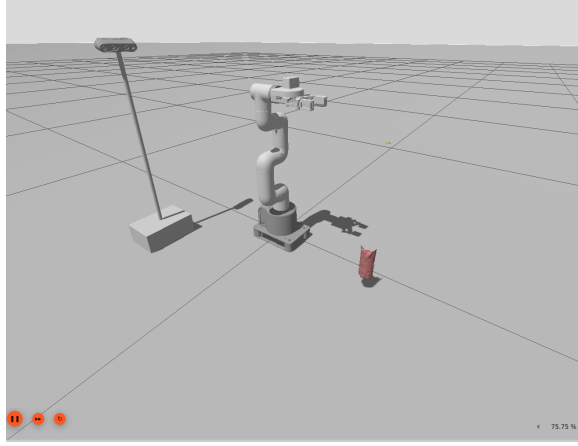


Figure 4: STL soup_can mesh imported into our gazebo environment.

7 Conclusion

This project successfully implemented a reconstruction-to-manipulation process through Ubuntu. The use of the cylindrical symmetry of the object and motion blur issues made the ROS2 SfM stage highly unstable, leading to scattered, spliced, and collapsed reconstructions. VisualSfM produced a usable dense model after manual cleaning, despite imperfections such as blurred texture and a missing bottom surface. After exporting the model to STL, the mesh was integrated into a UR5e MoveIt environment for collision-aware trajectory planning. Future work includes capturing the video at a lower-frame-rate, increasing parallax, obtaining top and bottom viewpoints, and integrating a functional grasping sequence.